

SIMATS ENGINEERING
Department of Computer Science &
Engineering ASSESSMENT TOOL 1- ANALYTICAL
PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	ANUSHYA DEVI V	Reg.No.:	192424013
Department:	AI&DS	Date:	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both

restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I ANUSHYA DEVI V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANUSHYA DEVI V

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Signed 8-bit Arithmetic Using 2's Complement

Given Numbers

- +45
- -23

Step 1: Convert to 8-bit Binary

+45

$$45 = 32 + 8 + 4 + 1$$

Binary: 00101101

-23

First convert +23 to binary.

23: 00010111

Take 2's complement.

1's complement: 11101000

Add 1: 11101001

Therefore,

$$-23 = 11101001$$

A. Addition (+45 + (-23))

00101101
+ 11101001

00010110

Carry out of MSB is discarded.

Result: 00010110

Decimal: 22

Therefore,

$$45 + (-23) = 22$$

Overflow Check

Overflow occurs only when:

- Positive + Positive = Negative
- Negative + Negative = Positive

Here,

Positive + Negative

Overflow **cannot occur**.

Result is correct.

B. Subtraction (+45 - (-23))

Subtracting a negative means adding a positive.

$$45 - (-23)$$

$$= 45 + 23$$

23 in binary 00010111

Addition

00101101
+00010111

01000100

Binary : 01000100

Decimal 68

Answer:

$$45 - (-23) = 68$$

Overflow Detection

Operands

Positive + Positive

Result is also Positive.

No overflow.

Processor Overflow Detection

Processor compares:

- Carry into sign bit
- Carry out of sign bit

If they are different,

Overflow = 1

Otherwise, Overflow = 0

In this example

Carry In = Carry Out

Hence, Overflow = No

Final Results:

Operation	Binary Result	Decimal	Overflow
$45 + (-23)$	00010110	22	No
$45 - (-23)$	01000100	68	No

2. Carry Look-Ahead Adder (CLA)

Suppose

A = 1011

B = 0110

Cin = 0

Step 1: Generate (G)

Formula

$$G_i = A_i \times B_i$$

Bit	A	B	G
0	1	0	0
1	1	1	1
2	0	1	0
3	1	0	0

Step 2: Propagate (P)

Formula

$$P_i = A_i \text{ XOR } B_i$$

Bit	P
0	1
1	0
2	1
3	1

Step 3: Carry Equations

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

All carries are computed simultaneously.

Ripple Carry Delay

Carries move one stage at a time.

$$\text{Bit}_0 \rightarrow \text{Bit}_1 \rightarrow \text{Bit}_2 \rightarrow \text{Bit}_3$$

Delay

$$4 \times \text{Gate Delay}$$

CLA Delay

Carries are generated together.

Delay

Approximately 2 Gate Delays

Comparison

Ripple Carry	CLA
Carry propagates serially	Carry generated in parallel
Slow	Fast
Delay increases with bits	Very small delay
Simple hardware	More complex hardware

Why CLA is Faster

- Parallel carry computation
- No waiting for previous carry
- Used in modern CPUs for high-speed arithmetic

3. Booth's Algorithm ($-6 \times +5$)

Given

Multiplicand $M = -6$

Multiplier $Q = +5$

Binary (4-bit)

$M = 1010$

$Q = 0101$

$A = 0000$

$Q_{-1} = 0$

Need 4 cycles.

Initial

$A = 0000$

$Q = 0101$

$Q_{-1} = 0$

Last bits

Q_0Q_{-1}

10

Rule

$10 \rightarrow A = A - M$

Then arithmetic right shift.

Cycle 2

Again check

Q_0Q_{-1}

Apply Booth rule.

Shift.

Cycle 3

Repeat

Add/Subtract depending on

01

10

00

11

Booth Rules

Q0Q-1	Operation
00	No operation
11	No operation
01	Add M
10	Subtract M

Final Result

-6×5

$=B-30$

Binary (8-bit)

11100010

Decimal

-30

Why Booth Algorithm is Efficient

Instead of adding many times,

Booth combines consecutive 1's into one addition and one subtraction.

Advantages

- Faster multiplication
- Fewer arithmetic operations
- Supports signed numbers directly
- Used in processors

4. Division ($13 \div 3$)

Binary

$$13 = 1101$$

$$3 = 0011$$

Expected answer

$$\text{Quotient} = 4$$

$$\text{Remainder} = 1$$

A. Restoring Division

Initial

$$A = 0000$$

$$Q = 1101$$

$$M = 0011$$

Step 1

Shift left

Subtract divisor

If negative

Restore

Set quotient bit = 0

Step 2

Shift

Subtract

Positive

Keep result

Quotient bit = 1

Repeat 4 times.

Final

$$\text{Quotient} = 0100$$

$$\text{Remainder} = 0001$$

B. Non-Restoring Division

Same initialization.

Instead of restoring,
 If remainder becomes negative,
 Next cycle performs addition.
 No immediate restoration.
 Final

Quotient = 0100

Remainder = 0001

Comparison

Restoring	Non-Restoring
Negative remainder restored immediately	Restoration postponed
More additions	Fewer operations
Slower	Faster
Simple implementation	More efficient
More clock cycles	Fewer clock cycles

Why NonRestoring is Preferred

- Fewer arithmetic operations
- Less execution time
- Better CPU performance
- Widely used in modern processors

5. IEEE 754 Floating-Point Representation

Given decimal number

−13.25

Step 1: Convert to Binary

Integer

13 = 1101

Fraction

0.25 = 0.01

Binary

1101.01

Normalize

1.10101×2^3

Single Precision (32-bit)

Sign

Negative

1

Exponent

Bias

127

Actual exponent

3

Stored exponent

$127+3=130$

Binary

10000010

Mantissa

Remove leading 1

101010000000000000000000

Final Representation

Sign

1

Exponent

10000010

Mantissa

101010000000000000000000

Complete 32 bits

1 10000010 101010000000000000000000

Double Precision (64-bit)

Bias

1023

Exponent

1026

Precision Loss

Occurs because:

- Mantissa has limited bits.
- Some decimal values (such as 0.1) cannot be represented exactly in binary.
- Rounding may slightly change the stored value.